Solving 2048: A Comparative Study of Classical Reinforcement Learning and LLM-Guided RLVR

Shriman Raghav
MS Robotics
Northeastern University
Boston, USA
srinivasan.shrim@northeastern.edu

Gautham Ramkumar
MS Robotics
Northeastern University
Boston, USA

***Abstract*—The puzzle game 2048 offers a compelling reinforcement learning benchmark that combines stochastic dynamics, a combinatorial state space, and delayed, sparse rewards. In this work we conduct a systematic comparison of five classical RL agents—Deep Q-Network (DQN), Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), Quantile Regression DQN (QR-DQN), and a discrete variant of Soft Actor-Critic (SAC)—against a 0.5-billion-parameter language model fine-tuned with Group Relative Policy Optimization (GRPO), a form of Reinforcement Learning with Verifiable Rewards (RLVR). Every classical agent is built on an identical three-layer convolutional feature extractor so that performance differences reflect the learning algorithm rather than architectural choices. Our results show that the off-policy DQN agent attains the highest average score of 3,401 and reaches the 512 tile in 21% of evaluation episodes, while the on-policy PPO and the distributional QR-DQN converge to approximately 1,000 on the same training budget. The GRPO-trained LLM, despite scoring only 196 on average, learns correct XML output formatting and all four directional moves within 500 gradient steps, highlighting RLVR's ability to simultaneously shape both format and strategy. We release a live terminal visualization framework and the complete codebase for reproducibility.**

***Index Terms*—reinforcement learning, deep Q-network, proximal policy optimization, quantile regression, soft actor-critic, GRPO, RLVR, language models, 2048**

I. Introduction

The tile-sliding puzzle 2048 was introduced in 2014 by Gabriele Cirulli and quickly attracted attention from the artificial intelligence community. On every turn the player selects one of four cardinal directions; all tiles slide in that direction, identical adjacent tiles merge (doubling their value and producing a score equal to the new tile), and a fresh 2 or 4 tile spawns at a uniformly random empty cell. The game terminates when no legal move remains.

Despite its compact board (4×4, 16 cells) the game poses several challenges for learning agents. The state space is vast—each cell can hold one of roughly 17 distinct values, yielding on the order of $17^{16} \approx 2.8 \times 10^{19}$ configurations before accounting for symmetries. Rewards are both sparse and delayed: a single merge of two 1024-tiles into a 2048-tile delivers a reward of 2048, yet the hundreds of preparatory moves that make this merge possible receive no direct credit. The stochastic tile spawns further complicate planning, as the agent cannot predict where the next obstacle will appear.

Classical approaches to 2048 span tabular n-tuple networks with temporal-difference (TD) learning [13], deep value-based methods such as DQN [3], [5], and policy-gradient methods including PPO [6]. Recently, a fundamentally different paradigm has emerged: Reinforcement Learning with Verifiable Rewards (RLVR), which trains large language models (LLMs) to generate chain-of-thought reasoning validated by deterministic reward functions [1], [2]. Although RLVR has yielded breakthroughs in mathematical reasoning, its application to spatial strategy games remains largely unexplored.

This paper asks three questions: **(Q1)** How do five modern RL algorithms compare on 2048 when architectural confounds are removed? **(Q2)** Can a small (0.5B-parameter) LLM acquire 2048 strategy through GRPO alone? **(Q3)** What trade-offs arise between classical RL's raw throughput and the LLM approach's interpretability? We make the following contributions:

1) A controlled benchmark of DQN, PPO, A2C, QR-DQN, and discrete SAC on a unified 2048 environment with a shared CNN backbone.
2) A novel GRPO training pipeline for an LLM 2048 agent with a five-component verifiable reward function.
3) An end-to-end inference pipeline for the LLM agent, including tokenization, constrained generation, and XML-based action parsing.
4) A live terminal visualization framework built with `rich` that supports real-time evaluation of any agent.

II. Related Work

A. Value-Based Methods for 2048

Szubert and Jaśkowski [13] introduced n-tuple networks trained with TD(λ) for 2048, learning position-specific value approximations without deep networks. Their approach reached the 2048 tile with high probability. Subsequent work by Jaśkowski showed that optimistic initialization of the value function encourages exploration of high-tile states, pushing the frontier to the 32768 tile. Saligram et al. [3] applied DQN with a 16-channel binary board encoding to 2048, establishing the CNN-based observation representation that we adopt in this work. Distributional variants—particularly QR-DQN [8] within the Horizon-DQN framework—have demonstrated the

ability to reach tile 4096 by capturing the heavy-tailed nature of 2048's return distribution.

B. Policy-Gradient and Actor-Critic Methods

PPO [6] and A2C [7] have been applied to board games and Atari, though their application to 2048 has received less attention than value-based methods. Haarnoja et al. [9] proposed SAC for continuous control; Christodoulou [10] later adapted it to discrete action spaces by replacing the Gaussian policy with a categorical distribution, making it applicable to 2048's four-action environment. Our work provides the first side-by-side evaluation of all five methods on 2048 with an identical feature extractor.

C. RLVR and Group Relative Policy Optimization

DeepSeek-R1 [1] demonstrated that GRPO—originally proposed for mathematical reasoning in DeepSeekMath [12]—can train LLMs to produce chain-of-thought solutions validated by external verifiers. GRPO side-steps the need for a critic network by normalizing rewards within a group of sampled completions, reducing memory requirements during training. Wen et al. [2] subsequently showed that RLVR implicitly incentivizes correct intermediate reasoning steps even when rewards are computed only from final answers. We extend this line of work to a spatial strategy game where the "final answer" is a directional move rather than a numerical result.

D. LLMs for Game Playing

Hu et al. [4] introduced lmgame-Bench, evaluating LLMs on games ranging from Tic-Tac-Toe to chess. Even frontier models such as GPT-4 struggle with spatial reasoning tasks that require mental simulation of board states. Our work differs in that we do not evaluate a frozen LLM; instead, we actively fine-tune it via GRPO to improve its 2048 play.

III. Methodology

A. Environment Design

We implement a custom 2048 game engine wrapped as a Gymnasium [17] environment. The environment exposes the standard `step`/`reset` interface with action space $\mathcal{A} = \{0, 1, 2, 3\}$ mapping to $\{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$.

1) State Representation: For CNN-based agents the raw board $B \in \mathbb{Z}_{\geq 0}^{4\times 4}$ is converted to a 16-channel binary tensor $\mathbf{x} \in \{0, 1\}^{16\times 4\times 4}$:

$$x_{k,i,j} = \mathbb{1}\left[B_{i,j} = 2^k\right], \quad k = 0, 1, \ldots, 15 \tag{1}$$

Channel 0 marks empty cells ($B_{i,j} = 0$); channel k marks cells containing tile 2^k, covering values from 2 up to 32768. This one-hot power-of-two encoding avoids the magnitude bias that would arise from feeding raw tile values into convolutional filters.

For the LLM agent, the board is serialized as a labeled ASCII grid with row and column headers, appended with the current score and maximum tile value.

2) Reward Modes: The environment supports three reward modes:

- **Score-delta**: the raw merge score produced by each move.
- **Log-score**: $\log_2(\text{score_delta} + 1)$, which compresses the reward scale for numerical stability.
- **Shaped**: score-delta augmented with milestone bonuses (+50 for reaching tile 256, +100 for 512, +200 for 1024, +500 for 2048).

All classical agents in this study use the score-delta mode for consistency. Invalid moves (those that do not change the board) receive a penalty of -1.

B. Shared CNN Architecture

To isolate the effect of the learning algorithm from the representation capacity, every classical agent uses the same convolutional feature extractor ϕ_θ:

$$\phi_\theta(\mathbf{x}) = \sigma(\mathbf{W}_3 \cdot \sigma(\mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 * \mathbf{x}))) \tag{2}$$

concretely consisting of three Conv2D layers (128 filters each, 2×2 kernel, stride 1, no padding) followed by ReLU activations (σ), a flatten operation, and a fully connected layer mapping to a 256-dimensional feature vector. Because the 4×4 input shrinks by one spatial dimension per convolution layer, the third convolution produces a $1 \times 1 \times 128$ output, which the fully connected layer projects to $\mathbb{R}^{256}$. Each algorithm then appends its own task-specific heads (Q-values, policy logits, value estimates) on top of this shared 256-dimensional representation.

C. Deep Q-Network (DQN)

DQN [5] parameterizes the action-value function $Q_\theta(s, a)$ as a neural network mapping the 256-dimensional features to four Q-values. The network is trained to minimize the mean squared Bellman error:

$$\mathcal{L}_{\text{DQN}}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(y - Q_\theta(s, a)\big)^2\Big] \tag{3}$$

where the target $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$ is computed using a frozen *target network* $\bar{\theta}$ that is periodically synchronized with the online network. This decoupling reduces the harmful correlation between the target and the prediction that causes training instability in naïve Q-learning.

Experience replay. Transitions $(s, a, r, s', \text{done})$ are stored in a circular buffer $\mathcal{D}$ of capacity 10^5. At each training step, a uniformly random mini-batch of 64 transitions is sampled, breaking the temporal correlation inherent in sequential game trajectories.

Exploration. The agent follows an ε-greedy policy: with probability ε it selects a uniformly random action, and otherwise it acts greedily on Q_θ. We linearly decay ε from 1.0 to 0.01 over 10^5 steps and hold it constant thereafter. This schedule provides aggressive early exploration followed by near-deterministic exploitation.

Gradient clipping. All gradients are clipped to a maximum norm of 1.0 to prevent catastrophic parameter updates during early training when the Q-value estimates are highly inaccurate.

D. Proximal Policy Optimization (PPO)

PPO [6] is an on-policy actor-critic method that constrains policy updates to a trust region defined by a clipping mechanism. The actor network $\pi_\theta(a|s)$ outputs a categorical distribution over four actions via a softmax head, and the critic $V_\theta(s)$ outputs a scalar state-value estimate. Both share the CNN backbone.

The policy is updated by maximizing the clipped surrogate objective:

$$\mathcal{L}_{\text{clip}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ g(\epsilon, \hat{A}_t)\big)\Big] \tag{4}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio between the current and previous policies, $\hat{A}_t$ is the Generalized Advantage Estimate (GAE), and $g(\epsilon, A) = \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\cdot A$ prevents the ratio from moving too far from unity. GAE combines multi-step returns with an exponential weighting controlled by λ=0.95:

$$\hat{A}_t = \sum_{l=0}^{T-t}(\gamma\lambda)^l\delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{5}$$

We set ϵ=0.2, collect 2048 steps per rollout, and train for 10 epochs per batch with mini-batch size 64.

E. Advantage Actor-Critic (A2C)

A2C [7] is the synchronous, single-worker variant of A3C. Like PPO it is on-policy and actor-critic, but it lacks the clipping mechanism, instead relying on a shorter rollout horizon and an entropy regularizer to stabilize learning.

The actor is updated via the REINFORCE gradient with a value baseline:

$$\nabla_\theta J(\theta) = \mathbb{E}_t\Big[\nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t\Big], \quad \hat{A}_t = R_t - V_\phi(s_t) \tag{6}$$

where $R_t = \sum_{k=0}^{n-1}\gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n})$ is the n-step bootstrapped return (we use n=5). The total loss sums three terms:

$$\mathcal{L}_{\text{A2C}} = -\underbrace{\mathbb{E}\Big[\log \pi \hat{A}\Big]}_{\text{policy}} + c_v \underbrace{\|R_t - V\|^2}_{\text{value}} - c_e \underbrace{\mathcal{H}[\pi]}_{\text{entropy}} \tag{7}$$

where c_v=0.5 balances the value loss and c_e=0.01 encourages exploration. Without the entropy term, A2C frequently collapses to a deterministic policy that repeatedly selects a single direction.

F. Quantile Regression DQN (QR-DQN)

Standard DQN models only the *expected* return $Q(s,a) = \mathbb{E}[Z(s,a)]$, discarding information about the shape of the return distribution. QR-DQN [8] instead learns the full distribution $Z(s,a)$ by approximating N fixed quantiles $\tau_1, \ldots, \tau_N$ of that distribution. We use N=50 quantiles, evenly spaced at $\tau_i = (2i-1)/(2N)$.

The network outputs an $|\mathcal{A}| \times N$ matrix, where entry (a,i) gives the predicted τ_i-th quantile $z_i(s,a)$ of $Z(s,a)$. Training minimizes the quantile Huber loss:

$$\mathcal{L}_{\text{QR}}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_j\big[\rho_{\tau_i}^{\kappa}(\delta_{ij})\big] \tag{8}$$

where $\delta_{ij} = r + \gamma z_j(s', a^*) - z_i(s,a)$ is the pairwise TD error between all quantile pairs, $a^* = \arg\max_{a'} \bar{Q}(s', a')$, and

$$\rho_\tau^\kappa(\delta) = |\tau - \mathbb{1}[\delta < 0]| \cdot \begin{cases} \frac{1}{2}\delta^2/\kappa & |\delta| \le \kappa \\ |\delta| - \frac{1}{2}\kappa & \text{else} \end{cases} \tag{9}$$

is the asymmetric Huber loss with threshold κ=1. At decision time, actions are chosen by the mean Q-value $Q(s,a) = \frac{1}{N}\sum_i z_i(s,a)$. The distributional representation captures reward variability: for example, a merge-heavy state may have a bimodal return distribution depending on where the random tile spawns.

G. Discrete Soft Actor-Critic (SAC)

Standard SAC [9] targets continuous actions via a Gaussian policy with a reparameterization trick. Since 2048 has a discrete four-action space, we implement the discrete adaptation of Christodoulou [10], which replaces the Gaussian with a categorical policy.

SAC solves the maximum-entropy RL objective:

$$J(\pi) = \sum_{t=0}^{T}\mathbb{E}_\pi[r_t + \alpha\mathcal{H}[\pi(\cdot|s_t)]] \tag{10}$$

where $\alpha > 0$ is a temperature parameter controlling the trade-off between reward maximization and policy entropy $\mathcal{H}$. Higher α produces more exploratory behavior; lower α drives the policy toward greedy exploitation.

Twin Q-networks. Two independent Q-networks Q_{ψ_1}, Q_{ψ_2} are maintained. The minimum of the two is used in both the actor and critic updates, mitigating the positive bias that arises from function approximation (clipped double Q-learning):

$$y = r + \gamma\sum_{a'}\pi_\theta(a'|s')\Big[\min_k Q_{\bar{\psi}_k}(s',a') - \alpha\log\pi_\theta(a'|s')\Big] \tag{11}$$

Actor update. The policy minimizes the KL divergence between the policy and the Boltzmann distribution induced by Q:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_s\left[\sum_a \pi_\theta(a|s)\Big(\alpha\log\pi_\theta(a|s) - \min_k Q_{\psi_k}(s,a)\Big)\right] \tag{12}$$

Automatic temperature tuning. Rather than fixing α by hand, we optimize it to maintain a target entropy $\bar{\mathcal{H}}$=0.5 $\log|\mathcal{A}|$:

$$\mathcal{L}_\alpha = -\alpha\cdot\mathbb{E}_s\big[\mathcal{H}[\pi(\cdot|s)] - \bar{\mathcal{H}}\big] \tag{13}$$

Target Q-networks are updated via Polyak averaging $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$ with τ=0.005.

H. LLM Agent: GRPO Training

1) Base Model and Quantization: We fine-tune Qwen2.5-0.5B-Instruct [15], a 494M-parameter causal language model, using 4-bit NormalFloat (NF4) quantization via the Unsloth framework [16]. Low-Rank Adaptation (LoRA) [11] adapters with rank r=32 and scaling α=32 are injected into all query,

key, value, output, gate, up, and down projection matrices, producing approximately 5.2 million trainable parameters (1.05% of the total).

2) Prompt Format: Each game turn generates a prompt pair. The **system prompt** describes the game rules, valid actions, and the required XML output schema. The **user prompt** contains the board as a labeled grid:

```
Board:
  C1 C2 C3 C4
R1  2  0  4  0
R2  0  8  2  0
R3  4  2 16  4
R4  0  2  4  2
Score: 148 | Max Tile: 16
```

The model must respond with:

```
<think>Reasoning here...</think>
<answer>DIRECTION</answer>
```

3) Group Relative Policy Optimization: GRPO [1], [12] eliminates the critic network that PPO and A2C require. For each prompt q the current policy π_θ generates a group of G=8 completions $\{o_1, \ldots, o_G\}$, each scored by the verifiable reward function. Advantages are normalized within the group:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon}, \quad \mu_G = \frac{1}{G}\sum_j r_j,\ \sigma_G = \sqrt{\frac{1}{G}\sum_j (r_j - \mu_G)^2} \tag{14}$$

This group normalization replaces the critic's state-dependent baseline with a prompt-specific, sample-based baseline. Because the reward function is deterministic, variance in r_i arises solely from the stochastic decoding of π_θ, ensuring a meaningful gradient signal as long as at least two distinct reward values appear within the group.

The policy gradient follows the clipped PPO-style objective with an additional KL penalty to prevent catastrophic forgetting of the pretrained language distribution:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_q\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\Big(L_t^{\text{clip}} - \beta D_{\text{KL}}\Big)\right] \tag{15}$$

where $L_t^{\text{clip}} = \min\big(r_t^i\hat{A}_i,\ \text{clip}(r_t^i, 1\pm\epsilon)\hat{A}_i\big)$, $r_t^i(\theta) = \pi_\theta(o_{i,t}|q, o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})$ is the per-token importance ratio, and β=0.1 weights the token-level KL divergence against the reference (initial) policy π_{ref}.

4) Multi-Component Reward Function: Each completion o_i receives a reward r_i computed as a weighted sum of five deterministically verifiable signals:

The quality and length components address the *cold-start problem*—early in training, the model's completions are too short and lack reasoning, producing zero reward variance across the group (since all G outputs are equally poor). By rewarding any non-trivial output, these components break the symmetry and provide gradient signal from the very first step.

TABLE I: GRPO Reward Components and Weights

Component	Weight	Computation
Format r_f	0.5	1 if output matches `<think>...</think><answer>...</answer>`, else 0
Direction r_d	0.5	1 if extracted direction $\in$ {UP, DOWN, LEFT, RIGHT}
Game r_g	2.0	Δscore/max_tile if valid move; -1 if invalid
Quality r_q	0.5	$\min(\lvert\text{thinking}\rvert/100, 1)$
Length r_ℓ	0.3	$\min(\lvert\text{completion}\rvert/200, 1)$

IV. Inference Pipeline

A. Classical Agent Inference

At inference time, each classical agent receives the 16-channel board encoding (equation 1), passes it through the shared CNN backbone, and computes the output in a single forward pass:

- **DQN / QR-DQN**: Select $a^* = \arg\max_a Q_\theta(s, a)$ with ε=0. For QR-DQN, Q-values are the mean of the 50 quantiles.
- **PPO / A2C**: Select $a^* = \arg\max_a \pi_\theta(a|s)$ (deterministic greedy).
- **SAC**: Select $a^* = \arg\max_a \pi_\theta(a|s)$ (greedy mode; stochastic mode uses categorical sampling).

All classical agents perform inference in under 4 ms per move on a single CPU core, owing to the lightweight nature of the 330K-parameter CNN. No special hardware is required for deployment.

B. LLM Inference Pipeline

The LLM inference pipeline is substantially more complex and involves four stages:

1. Prompt Construction. The current board state is rendered as a labeled text grid and concatenated with the system prompt. The combined prompt is tokenized using the Qwen2 tokenizer with a chat template, yielding approximately 180–220 input tokens per turn.

2. Autoregressive Generation. The model generates a completion of up to 256 tokens using the following sampling parameters: temperature T=0.7, nucleus sampling with p=0.95, and repetition penalty 1.0. The prompt is prefilled with the "`<think>`" token to guide the model into the expected output format. Generation terminates upon encountering the end-of-sequence token or reaching the maximum length.

3. Response Parsing. A two-stage parser extracts the action from the generated text. The primary parser uses regular expressions to locate the `<answer>` XML tag and extracts the direction keyword. If primary parsing fails (e.g., malformed XML), a fallback parser scans the entire response for any occurrence of a direction keyword (UP, DOWN, LEFT, RIGHT), selecting the last match. If both parsers fail, the agent selects a uniformly random valid action.

4. Action Execution. The extracted direction is mapped to the integer action code and executed in the environment. Invalid moves (those that do not change the board) are counted

TABLE II: Inference Latency Per Move

Agent	ms/move	Hardware	Parameters
DQN	3.6	CPU	330K
PPO	<1	CPU	330K
A2C	<1	CPU	330K
QR-DQN	4.2	CPU	430K
SAC	<1	CPU	330K
LLM (GRPO)	1,500	GPU	494M

TABLE III: Classical Agent Performance (Last 100 Training Episodes)

Agent	Avg Score	Max Score	Max Tile	256+	512+
DQN	$3{,}401 \pm 1{,}926$	8,708	512	69%	21%
PPO	$1{,}090 \pm 490$	2,988	256	4%	0%
QR-DQN	933 ± 485	2,564	256	3%	0%
A2C		*Training in progress*			
SAC		*Training in progress*			

separately; if 10 consecutive invalid moves occur, the game is terminated early.

The complete pipeline requires approximately 1,500 ms per move on an NVIDIA RTX 4060 GPU (with the 4-bit quantized model), making it roughly 400× slower than the classical agents. This latency is dominated by the autoregressive token generation (stage 2), which must sequentially produce 50–200 tokens.

C. Runtime Comparison

Table II summarizes the per-move inference cost across all agents.

V. Experiments

A. Setup

All experiments were conducted on a workstation equipped with an NVIDIA RTX 4060 GPU (8 GB VRAM) and an AMD Ryzen CPU. Classical agents were trained for 500K environment steps (DQN, QR-DQN, SAC) or 1M steps (PPO, A2C) using the score-delta reward mode. DQN and SAC use our custom PyTorch implementations; PPO and A2C use Stable-Baselines3 [14]; QR-DQN uses SB3-Contrib. The LLM agent was trained for 500 GRPO steps on a dataset of 5,000 game-state prompts with $G{=}8$ completions per prompt. No hyperparameter search was conducted; all values follow the recommendations in the cited papers or the respective library defaults.

B. Classical RL Results

Table III presents the performance of the three fully trained classical agents evaluated over their last 100 training episodes.

DQN dominance. DQN substantially outperforms both on-policy methods. We attribute this to two factors: (i) experience replay enables DQN to revisit high-scoring trajectories thousands of times, amplifying the learning signal from rare events such as 512-tile merges; and (ii) the ε-greedy schedule provides structured exploration that gradually transitions from random play to learned strategy. On-policy methods discard each trajectory after a single policy update, making them less sample-efficient in this sparse-reward setting.

fig_dqn_curves.png

Fig. 1: DQN training curves over 500K steps. The agent shows steady score improvement with max tile reaching 512 after approximately 200K steps.

TABLE IV: LLM Agent Performance After 500 GRPO Steps

Metric	Value
Average score	196
Maximum score	432
Maximum tile	32
Format compliance	100%
Direction diversity	4/4 directions used
Avg moves/game	28
Inference speed	∼1,500 ms/move

QR-DQN underperformance. Despite its richer distributional representation, QR-DQN did not surpass standard DQN at 500K steps. The 50-quantile output head increases the network's output dimensionality from 4 to $4 \times 50{=}200$, requiring more samples to accurately fit the distribution. We hypothesize that with a larger training budget (e.g., 2M steps) or prioritized experience replay, QR-DQN would eventually surpass DQN by better capturing the heavy-tailed reward structure.

Fig. 1 and Fig. 2 show the training curves for DQN and QR-DQN respectively.

C. LLM-RLVR Results

The GRPO-trained LLM was evaluated over 20 games after 500 training steps. Table IV summarizes the key metrics.

Format learning. The most striking finding is that the LLM achieved 100% compliance with the XML output schema (`<think>...</think><answer>...</answer>`) by step 500. At step 100, the model had only a ∼60% format

fig_qrdqn_curves.png

Fig. 2: QR-DQN training curves over 500K steps. Performance plateaus around 900 average score, with max tile capped at 256 throughout training.

fig_llm_curves.png

Fig. 3: LLM agent evaluation curves showing score progression and tile distribution over evaluation episodes.

TABLE V: Cross-Method Comparison

	DQN	**PPO**	**QR-DQN**	**LLM**	**Rand.**
Avg Score	3,401	1,090	933	196	$\sim$1,100
Max Tile	512	256	256	32	128
Interpretable	No	No	No	Yes	N/A
ms/move	3.6	$<$1	4.2	1,500	$<$0.1
Train Steps	500K	1M	500K	500	0
Parameters	330K	330K	430K	494M	0
Learning Type	Off	On	Off	GRPO	—

compliance rate. This confirms that GRPO can effectively teach structural constraints through verifiable rewards without supervised formatting examples.

Direction diversity. At step 100, the model exhibited a strong bias toward RIGHT and LEFT moves, rarely selecting UP or DOWN. By step 500, all four directions were utilized, indicating that the game reward component successfully penalized the collapse to a two-direction policy.

Reasoning evolution. We observed qualitative improvements in the model's chain-of-thought reasoning. Early-stage outputs contained random directional guesses ("*I'll go RIGHT*"), while later outputs included board-aware justifications ("*The 8 and 8 in R2 can merge if I move LEFT. Moving LEFT will also push the 4 tiles together in R3.*").

Fig. 3 shows the LLM evaluation curves.

D. Comparative Analysis

The gap between the best classical agent (DQN, 3,401) and the LLM (196) spans more than an order of magnitude. However, this comparison should be contextualized: the LLM processed only 500$\times$8=4,000 effective game states during training, while DQN processed 500K transitions with replay. Moreover, the LLM must simultaneously learn (i) XML output syntax, (ii) spatial board parsing from text, and (iii) game strategy—three tasks that the classical agents never face because their observation encoding and output format are hardcoded.

A surprising finding is that PPO and the random baseline achieve similar average scores ($\sim$1,000–1,100). This suggests that 1M training steps is insufficient for PPO to substantially outperform random play in this environment, likely due to the on-policy sample inefficiency and the high variance of 2048 returns.

VI. Discussion

A. Off-Policy Advantage in Sparse-Reward Games

Our results provide strong evidence that off-policy learning (DQN) significantly outperforms on-policy methods (PPO, A2C) in environments characterized by sparse, high-variance rewards. The experience replay mechanism acts as a natural curriculum: as the agent improves, its replay buffer accumulates an increasingly rich distribution of high-scoring trajectories that inform future updates. In contrast, PPO and A2C process each trajectory exactly once and must rely on the current policy to generate informative training data—a chicken-and-egg problem in the early stages of learning.

B. Distributional RL: Promise and Challenges

QR-DQN's performance comparable to (rather than exceeding) standard DQN contradicts the intuition that distributional

methods should excel in stochastic environments. We identify three possible explanations: (1) the 50-quantile representation increases the effective output dimensionality by 12.5×, requiring proportionally more training data; (2) the unimodal reward structure at low tile values (most merges produce similar-magnitude rewards) reduces the information advantage of a distributional model; and (3) at 500K steps, the agent has not yet encountered enough high-tile states to benefit from risk-sensitive decision making.

C. LLM as a Spatial Reasoner

The GRPO-trained LLM demonstrates that verifiable rewards can teach both format and strategy simultaneously. However, the model's primary bottleneck is *spatial reasoning from text*: it must mentally simulate the effect of sliding tiles in a direction, a task that requires multi-step symbolic manipulation of a 2D array encoded as a flat token sequence. This is fundamentally harder than the vectorized matrix operations that the CNN performs in a single forward pass. We conjecture that incorporating a visual token encoder or a structured board representation could substantially improve the LLM's performance.

D. Practical Trade-offs

The classical agents offer superior performance at dramatically lower computational cost (3.6 ms vs. 1,500 ms per move). However, the LLM provides a unique advantage: *interpretable reasoning*. Each move is accompanied by a natural-language explanation that can be inspected, debugged, and corrected. In applications where explainability is valued—such as educational tools or human-AI collaboration—this 400× latency penalty may be acceptable.

E. Limitations

Our study has several limitations. First, the training budgets are modest by modern standards; longer training for all agents would clarify convergence behavior. Second, we use a single random seed per agent, precluding statistical significance tests across multiple runs. Third, the LLM's 0.5B parameter scale is at the lower end of practical language models; 7B+ models may exhibit qualitatively different behavior. Finally, we do not explore hybrid approaches (e.g., LLM-guided MCTS) that could combine the interpretability of the LLM with the efficiency of classical search.

VII. Conclusion

We have presented a systematic comparison of five classical RL agents and an LLM-RLVR agent on the 2048 puzzle game. Our controlled experimental setup—with a shared CNN backbone, identical environment, and consistent reward function—isolates the effect of the learning algorithm from confounding architectural differences. Key findings include:

- DQN achieves the highest performance (avg. score 3,401, tile 512 in 21% of games), benefiting from off-policy experience replay in this sparse-reward environment.
- On-policy methods (PPO, A2C) struggle with sample efficiency, performing comparably to the random baseline at the tested training budget.
- QR-DQN's distributional approach does not confer an advantage at 500K steps but may benefit from longer training.
- The GRPO-trained LLM learns correct XML formatting and directional diversity within 500 steps, demonstrating RLVR's ability to shape both output structure and task strategy.
- Classical agents are ∼400× faster at inference, but the LLM provides unique interpretability through chain-of-thought reasoning.

Future work includes scaling to larger LLMs (7B+ parameters), implementing curriculum learning with progressive board complexity, exploring hybrid LLM-MCTS approaches, and conducting ablation studies on each GRPO reward component.

Acknowledgment

This project was completed as part of CS 5180 (Reinforcement Learning) taught by Professor Robert Platt at the Khoury College of Computer Sciences, Northeastern University. All experiments were conducted on a single NVIDIA RTX 4060 GPU (8 GB VRAM).

References

[1] D. Guo *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025.

[2] Y. Wen *et al.*, "RLVR implicitly incentivizes correct reasoning," *arXiv:2506.14245*, 2025.

[3] P. Saligram, T. Bhathal, and R. Manihani, "2048: Reinforcement learning in a delayed reward environment," *arXiv:2507.05465*, 2025.

[4] L. Hu *et al.*, "lmgame-Bench: How good are LLMs at playing games?" *arXiv:2505.15146*, 2025.

[5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[7] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. ICML*, pp. 1928–1937, 2016.

[8] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. AAAI*, 2018.

[9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. ICML*, 2018.

[10] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv:1910.07207*, 2019.

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized language models," in *Proc. NeurIPS*, 2024.

[12] Z. Shao *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv:2402.03300*, 2024.

[13] M. Szubert and W. Jaśkowski, "Temporal difference learning of n-tuple networks for the game 2048," in *Proc. IEEE CIG*, 2014.

[14] A. Raffin *et al.*, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[15] Qwen Team, "Qwen2.5: A party of foundation models," *arXiv:2412.15115*, 2024.

[16] D. Han and M. Han, "Unsloth," Available: https://github.com/unslothai/unsloth, 2024.

[17] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv:2407.17032*, 2024.